

Timeline

Updated: 2018-09-06

A component that combines timing measurements from an observed `LocalClock` or `GlobalClock` and any `AreaClock` within which it is.



This component should be attached to any `GameObject` that should be affected by Chronos.

Properties

```
TimelineMode mode { get; set; }
```

Determines what type of clock the timeline observes.

Value	Description
Local	The timeline observes a <code>LocalClock</code> attached to the same <code>GameObject</code> .
Global	The timeline observes a <code>GlobalClock</code> referenced by <code>globalClock</code> .

```
string globalClockKey { get; set; }
```

The key of the `GlobalClock` that is observed by the timeline. This value is only used for the `Global` mode.

```
bool rewindable { get; set; }
```

Determines whether the timeline should support rewind.

```
float recordingDuration { get; }
```

The maximum duration in seconds during which snapshots will be recorded. Higher values offer more rewind time but require more memory.

```
float recordingInterval { get; }
```

The interval in seconds at which snapshots will be recorded. Lower values offer more rewind precision but require more memory.

```
float availableRewindDuration { get; }
```

Returns the available rewind duration in seconds.

```
Clock clock { get; }
```

The clock observed by the timeline.

```
float timeScale { get; }
```

The time scale of the timeline, computed from all observed clocks. For more information, see `Clock.timeScale`.

```
float deltaTime { get; }
```

The delta time of the timeline, computed from all observed clocks. For more information, see `Clock.deltaTime`.

```
float smoothDeltaTime { get; }
```

A smoothed out delta time. Use this value if you need to avoid spikes and fluctuations in delta times. The amount of frames over which this value is smoothed can be adjusted via `smoothingDeltas`.

```
static int smoothingDeltas { get; set; } = 5
```

The fixed delta time of the timeline, computed from all observed clocks. For more information, see `Clock.fixedDeltaTime`.

```
float time { get; }
```

The time in seconds since the creation of this timeline, computed from all observed clocks. For more information, see `Clock.time`.

```
float unscaledTime { get; }
```

The unscaled time in seconds since the creation of this timeline. For more information, see `Clock.unscaledTime`.

```
TimeState state { get; }
```

Indicates the state of the timeline.

Value	Description
Accelerated	Time is accelerated (time scale > 1).
Normal	Time is in real-time (time scale = 1).
Slowed	Time is slowed ($0 < \text{time scale} < 1$).
Paused	Time is paused (time scale = 0).
Reversed	Time is reversed (time scale < 0).

Methods

`void SetRecording(float duration, float interval)`

Sets the recording duration and interval in seconds.



This will reset the saved snapshots.

`void ResetRecording()`

Resets the saved snapshots.

`int EstimateMemoryUsage()`

Estimate the memory usage in bytes from the storage of snapshots for the current recording duration and interval.

`void ReleaseFrom(AreaClock areaClock)`

Releases the timeline from the specified area clock's effects.

```
void ReleaseFromAll()
```

Releases the timeline from the effects of all the area clocks within which it is.

```
Coroutine WaitForSeconds(float seconds)
```

Suspends the coroutine execution for the given amount of seconds. This method should only be used with a yield statement in coroutines.



There is currently no built-in support for rewindable coroutines due to limitations in the way .NET enumerators work. This feature is being considered for a future release if a workaround can be found. For now, if time is going backward, `WaitForSeconds` will simply never finish.

```
void CacheComponents()
```

The components used by the timeline are cached for performance optimization. If you add or remove built-in Unity components on the `GameObject`, you need to call this method to update the timeline accordingly.

```
void Reset()
```

Attempts a full manual reset of the timeline. When using game object pooling, you should call this method after despawning or before spawning.

Previous



AreaClock

A Clock that affects every Timeline within its collider by multiplying its time scale with that of their observed clock.

Next

TimelineChild



A component that transfers the effects of any ancestor Timeline in the hierarchy to the current `GameObject`.

Was this article helpful?

Be the first to vote!

👍 Yes, helpful

👎 No, not for me



PRODUCTS

LUDIQ

Contact

Blog

Copyright © 2022 Ludiq. All Rights Reserved. • • Terms of Service • Icons by IconMonstr and FatCow